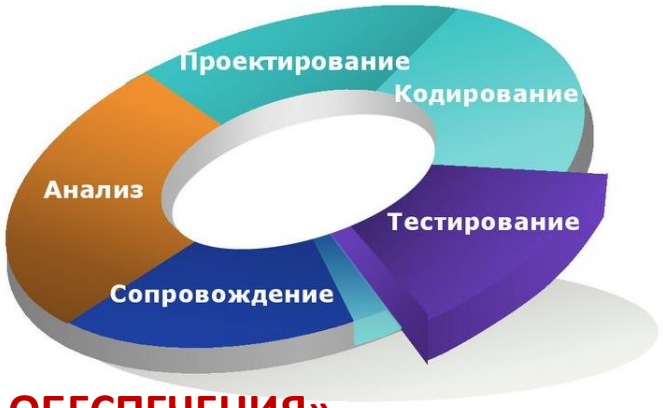


«ТЕХНОЛОГИЯ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ»

Унифицированный язык моделирования UML

:

09.02.07

Информационные системы и программирование

Язык объектно-ориентированного моделирования UML

Моделирование современных (объектно-ориентированных) систем производится с помощью **Unified Modeling Language** (UML) — унифицированного языка моделирования. UML является языком для определения, визуализации, создания и документирования компонентов систем ПО, а также для бизнес-моделирования и других систем, не являющихся ПО. UML представляет собрание лучших технических методов, которые доказали свою эффективность в моделировании больших и сложных систем

Спецификация UML определяет шесть видов диаграмм: state structure (структуры состояний), use case (сценариев использования), interaction (взаимодействия), statechart (состояния), collaboration (сотрудничества), activity (деятельности), а также диаграммы implementation (выполнение).


диаграмма вариантов использования
Исходная концептуальная модель.
Описывает функциональное назначение системы
диаграмма классов
представление структуры
диаграмма кооперации
взаимодействие классов структурного характера
диаграмма компонентов
архитектура программной реализации
Статическое представление
Интегрированная модель сложной системы
диаграмма состояний
процесс изменения состояний
диаграмма деятельности
частный случай диаграммы состояний
диаграмма последовательности
временной аспект поведения
диаграмма развертывания
представление компонентов программы, существующих на этапе выполнения
Динамическое представление
Концептуальное и логическое представление
Физическое представление

Интегрированная модель сложной системы в нотации UML

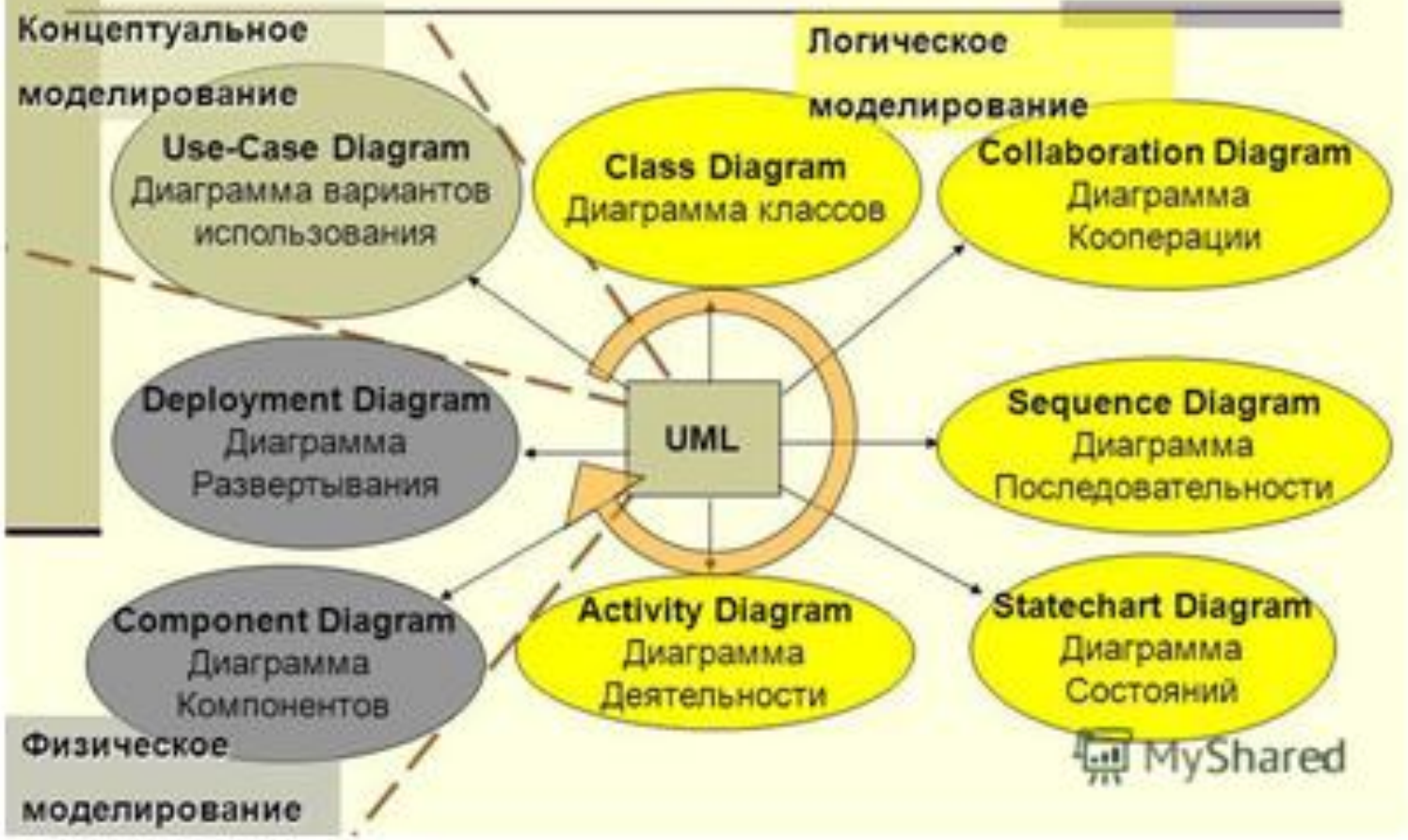
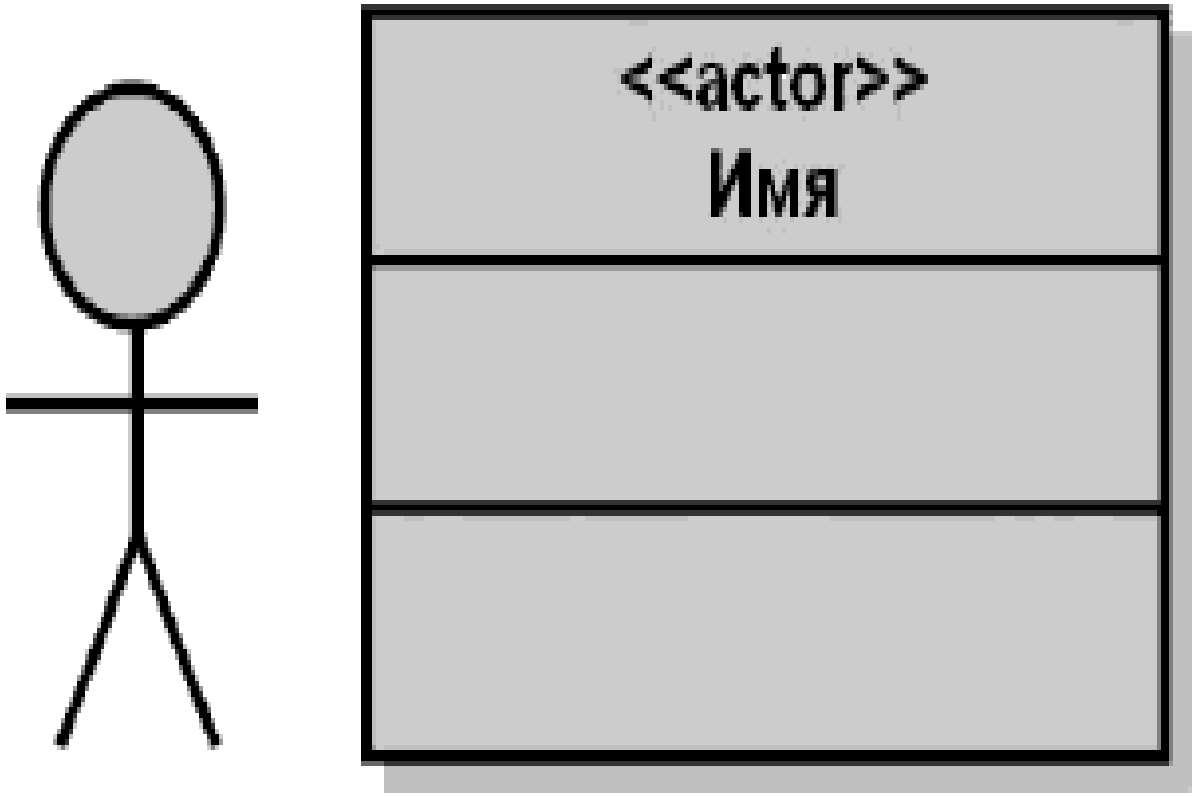


Диаграмма прецедентов (use case diagram)

Эктор (actor) - ***это множество логически связанных ролей, исполняемых при взаимодействии с прецедентами или сущностями (система, подсистема или класс).***



Прецедент (use case) - ***описание множества последовательных событий (включая варианты), выполняемых системой, которые приводят к наблюдаемому эктором результату.***



Диаграммы прецедентов относятся к той группе диаграмм, которые представляют динамические или поведенческие аспекты системы.
Это *средство для достижения взаимопонимания между разработчиками, экспертами и конечными пользователями* продукта.

Цели создания диаграмм прецедентов:

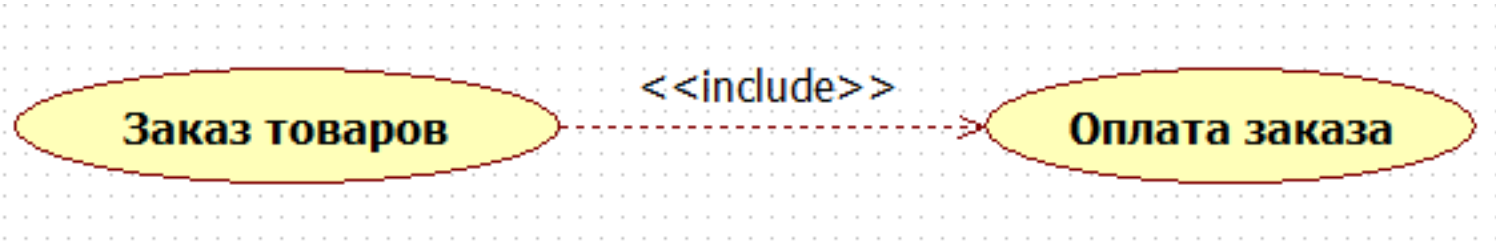
- определение границы и контекста моделируемой предметной области на ранних этапах проектирования;
- формирование общих требований к поведению проектируемой системы;
- разработка концептуальной модели системы для ее последующей детализации;
- подготовка документации для взаимодействия с заказчиками и пользователями системы.


плата за обучение
получение стипендии
получение кредита
переход на бесплатное обучение
студент
финансовый отдел
перевод с курса на курс
администратор
составление расписания курса
проведение занятий курса
инструктор
записаться на курс
отказаться от курса
посетить занятия курса
закончить курс
отказаться от обучения
окончить университет
доведение до ведома студентов
секретарь
<<extend>>
<<extend>>
сообщение графика оплаты
<<extend>>
учебная часть
сообщение расписания
сообщение списка выбранных курсов
написание заявки на грант
исследователь

Между прецедентами также возможны взаимоотношения, которые описываются отношениями двух типов:
включения и расширения (дополнения).

Включение (include) говорит о том, что исходный прецедент явным образом включает в себя поведение целевого.
Включение создается, когда один прецедент использует другой.
При этом исполнение базового прецедента невозможно без исполнения используемого.
Изображается включение в виде пунктирной стрелки с надписью <<include>>, которая *направлена от базового элемента к используемому*.

Пример В системе заказов магазина «Style» невозможен заказ товаров без оплаты.



Расширение (extend) показывает, что целевой прецедент расширяет поведение исходного.
Используемый прецедент выполняется не всегда вместе с базовым, а только при выполнении дополнительных условий, таким образом, расширяя функциональность базового элемента.
Изображается расширение пунктирной стрелкой с надписью **<<extend>>,** направленной *от используемого варианта использования к базовому*.

Пример При заказе товаров в системе заказов магазина «Style» покупатель может изменить содержание корзины перед тем, как оформить заказ окончательно, а может оставить корзину без изменений.
Изменение корзины – это опция, которую на диаграмме вариантов использования можно изобразить с помощью расширения.

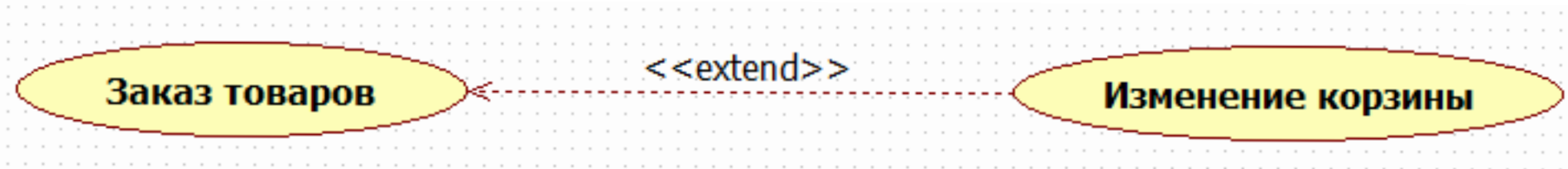


Диаграмма активности (деятельности, activity diagram)

Диаграммы деятельности удобно применять для визуализации **алгоритмов**

Нотация UML предлагает пять представлений системы:

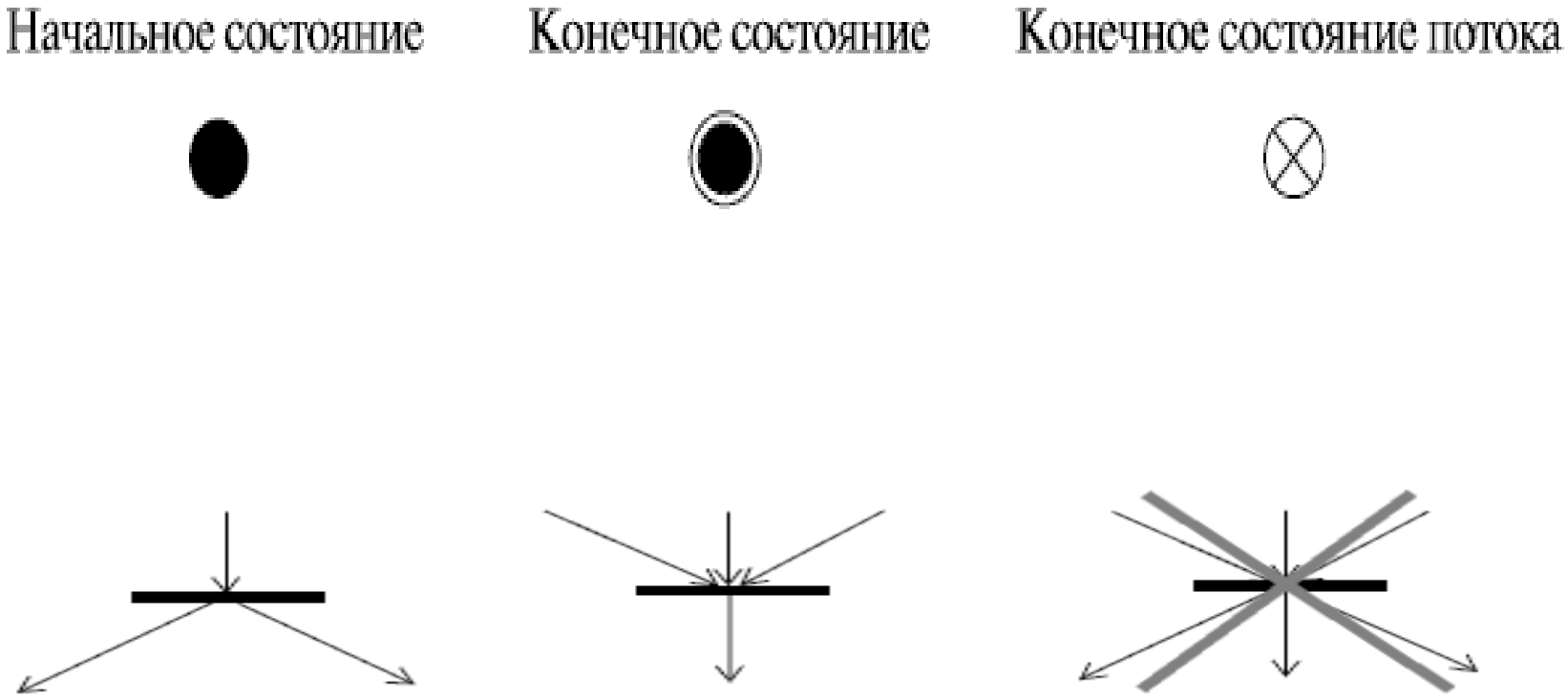
- Вид системы с точки зрения *прецедентов*.
- Вид с точки зрения *проектирования*.
- Вид с точки зрения *процессов*.
- Вид с точки зрения *развертывания*.
- Вид с точки зрения *реализации*.

каждый из перечисленных способов представления системы может содержать **последовательности действий, которые могут быть описаны с помощью алгоритмов**

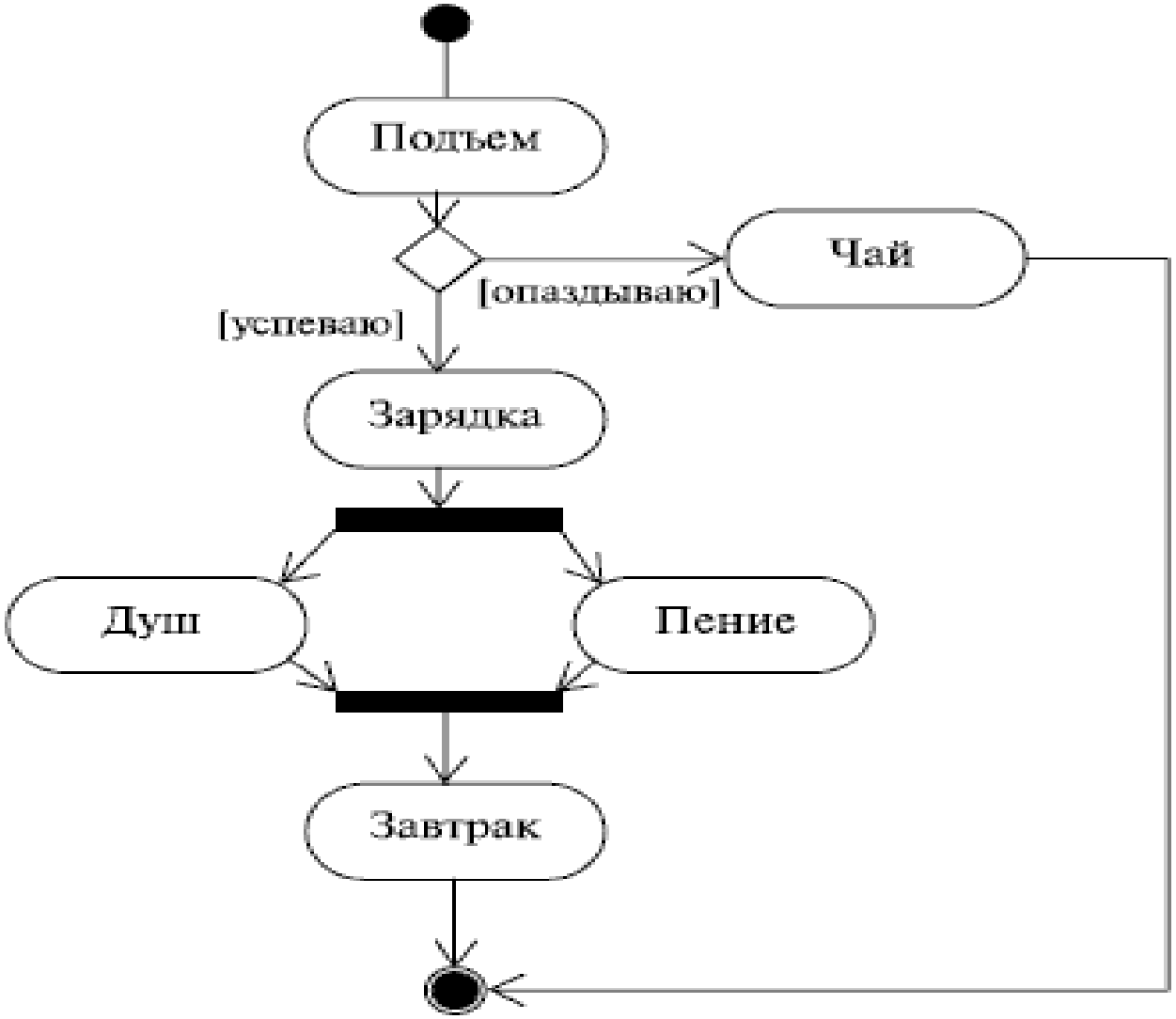
На диаграмме деятельности представлены **переходы** потока управления от одной деятельности к другой

Диаграммы деятельности позволяют моделировать сложный **жизненный цикл объекта**, с переходами из одного состояния (деятельности) в другое.

Диаграмма деятельности может быть присоединена к любому элементу модели, имеющему динамическое поведение



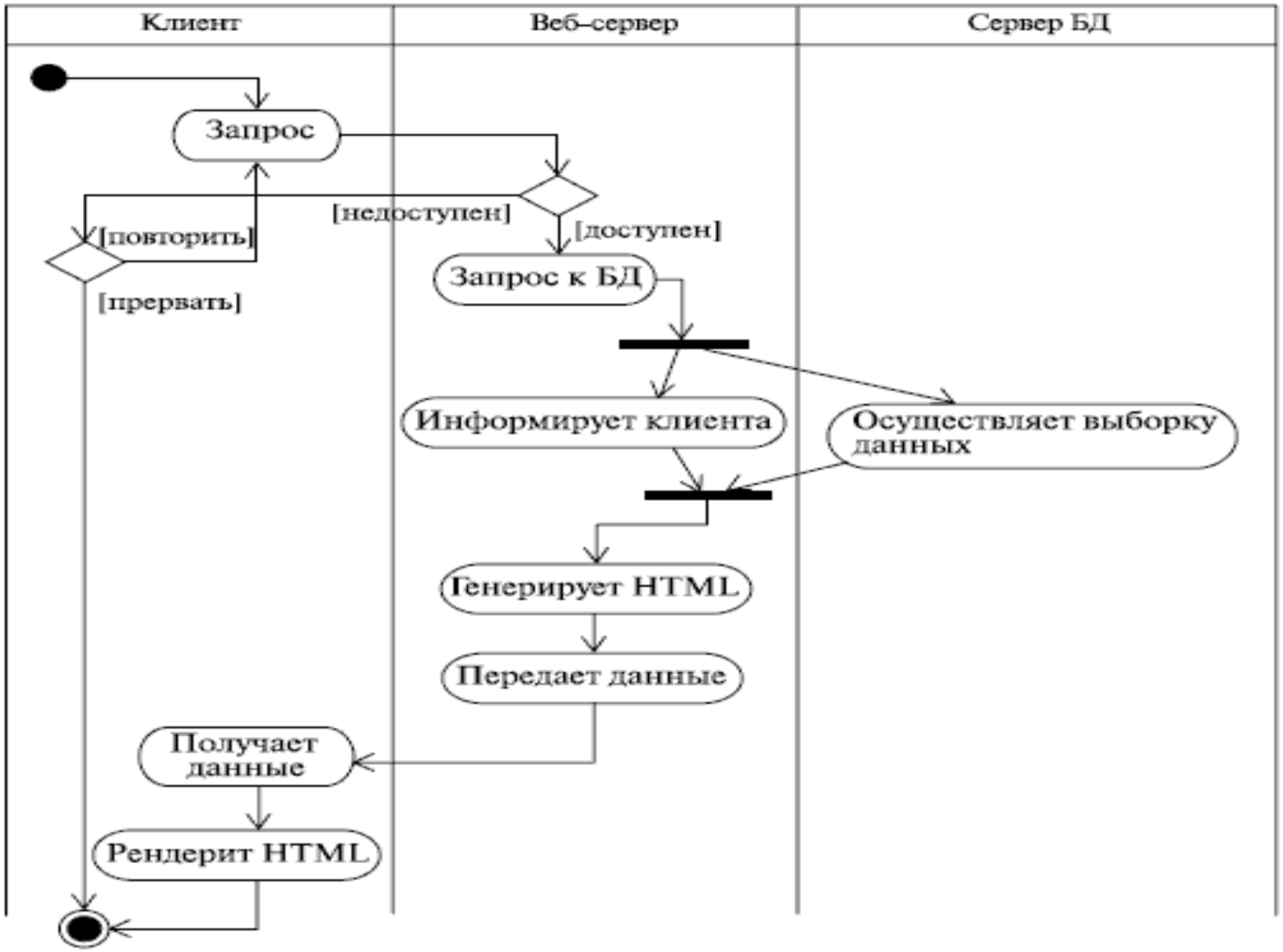
несколько потоков управления сливаются в один или один *поток* разделяется на несколько.



параллельное пение
и принятие душа



оформление заказа в интернет-магазине



работа с веб-приложением, которое решает некую задачу в удаленной базе данных. активности на этой диаграмме «как бы» разбросаны *по* трем дорожкам, каждая из которых соответствует поведению одного из трех объектов - клиента, веб-сервера и сервера баз данных. Благодаря этому легко определить, каким из объектов выполняется каждая из активностей

Деятельность **- это протяженное *по* времени составное действие. То есть *составленное* из более простых действий. Эти простые (атомарные) действия (последовательность их выполнения) изображают внутри деятельности в виде *диаграммы* внутри другой.**

Пример"вложенных" диаграмм.

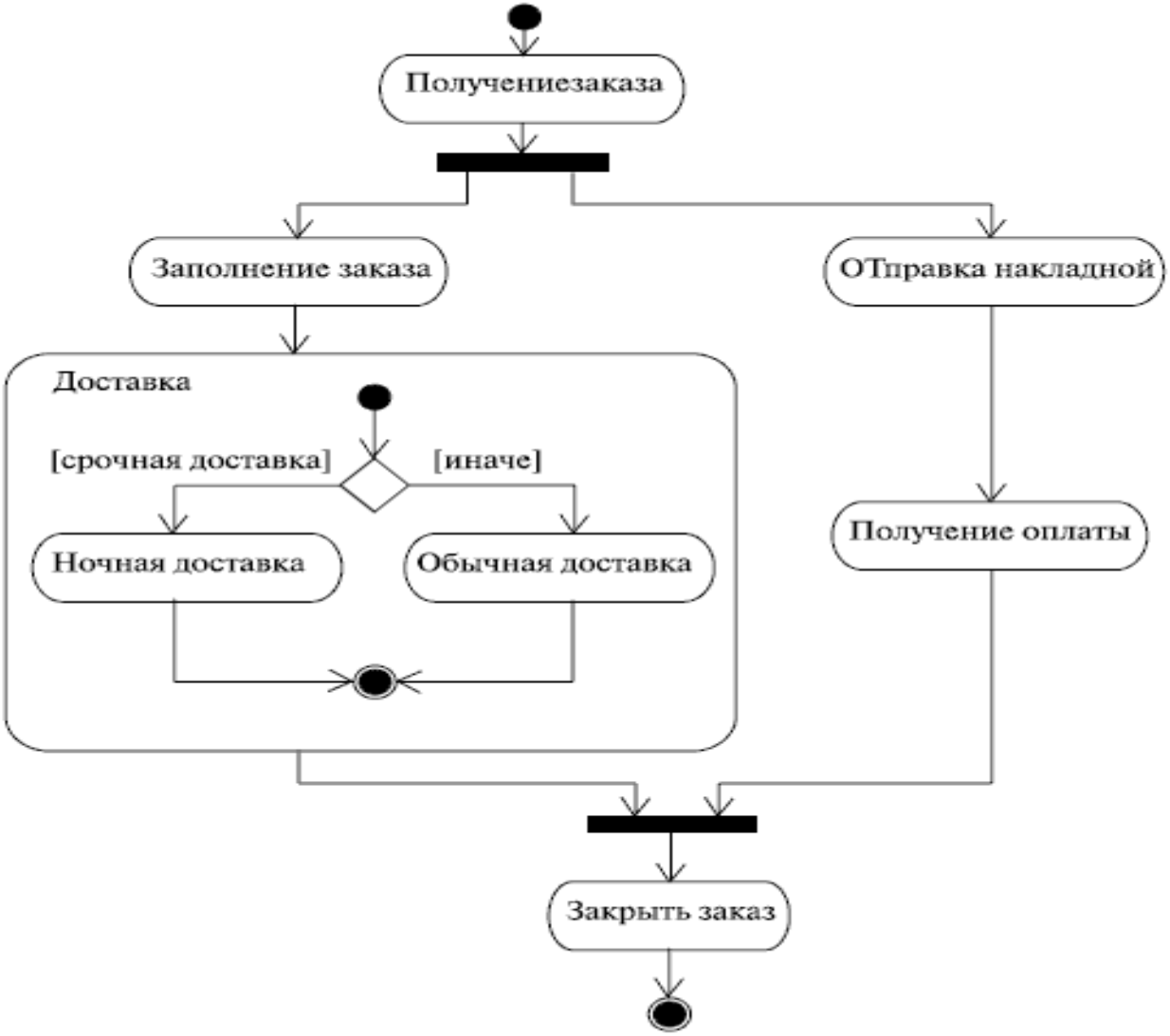

Получениезаказа
Заполнение заказа
ОТправка накладной
Доставка
[срочная доставка]
[иначе]
Ночная доставка
Обычная доставка
Получение оплаты
Закрыть заказ



Диаграмма описывает высадку пассажиров самолета, достигших пункта назначения, и посадку новых пассажиров

Выводы

- Диаграммой деятельности можно дополнить любой элемент модели, имеющий динамическое поведение.
- Диаграммы деятельности являются частным случаем диаграммы состояний.
- В отличие от блок-схем, диаграммы деятельности могут отображать одновременно выполняемые действия.
- На диаграммах активности можно использовать дорожки, распределяющие деятельности в соответствии с ролями (объектами), их выполняющими.
- Траектория объекта позволяет показать объекты, относящиеся к деятельности, и моменты переходов этих объектов из одного состояния в другое.
- Сложные деятельности можно дополнительно детализировать, разбив на действия и изобразив "диаграмму в диаграмме".
- Диаграммы деятельностей можно использовать для проектирования процессов (например, бизнес-процессов) или операций (вычислений). Во втором случае UML выступает в роли визуального языка программирования.

Диаграмма классов (class diagram)

Класс (class) - описание множества объектов с общими атрибутами, операциями, отношениями и семантикой.

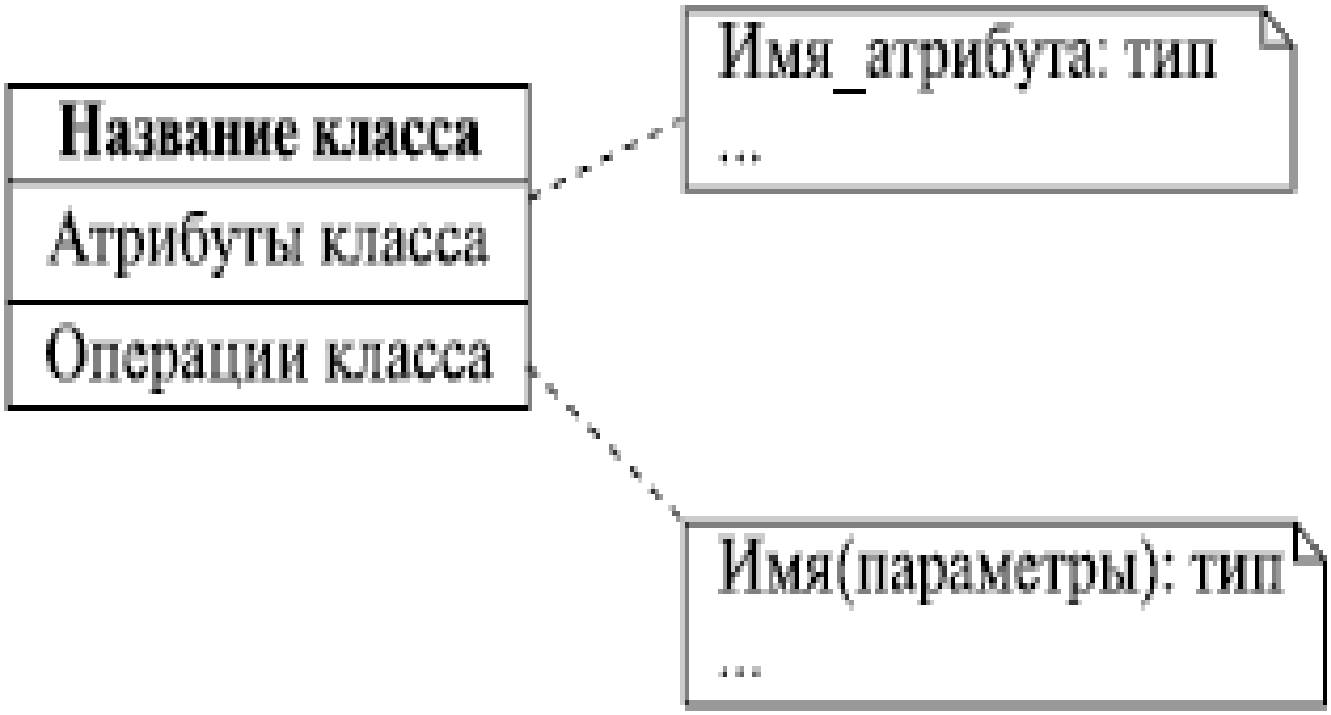
Классы - *это строительные блоки любой объектно-ориентированной системы*.

Диаграмма классов - это *набор статических*, декларативных *элементов модели*.

Классы используются в **процессе анализа предметной области** для составления *словаря предметной области* разрабатываемой системы.

Это могут быть как абстрактные понятия предметной области, так и классы, на которые опирается разработка и которые описывают программные или аппаратные сущности.

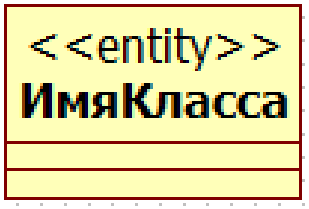
Информация с диаграммы классов напрямую отображается в исходный код приложения - в большинстве существующих инструментов UML-моделирования возможна ***кодогенерация*** *для определенного языка программирования (Java, C++).*

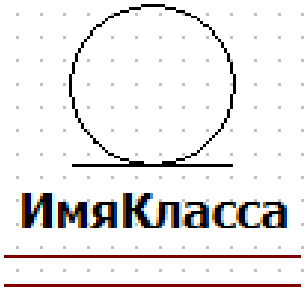

Название класса
Атрибуты класса
Операции класса
Имя_атрибута: тип
...
Имя(параметры): тип
...

***Стереотип** – это механизм, позволяющий категоризировать классы. Он используется для создания нового типа элемента, в данном случае нового типа класса.*

- сущность (entity),

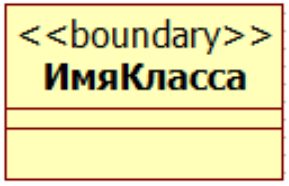
Класс-сущность содержит информацию, хранимую постоянно





- граница (boundary)

Граничными классы - классы, расположенные на границе системы со всем остальным миром, и т.о. они обеспечивают взаимодействие между окружающей средой и внутренними элементами системы.





- управление (control).

Управляющий класс отвечает за координацию действий других классов. Они служат для моделирования последовательного поведения одного или нескольких прецедентов и координации событий, реализующих заложенное в них поведение.





Инкапсуляция – упаковка данных и функций в единый компонент.

- механизм языка, позволяющий ограничить доступ одних компонентов программы к другим;
- языковая конструкция, позволяющая связать данные с методами, предназначенными для обработки этих данных.

- Обеспечивает защиту отдельных элементов объекта, не затрагивающих существенных характеристик его как целого.
- В программировании инкапсуляция реализуется с помощью ***модификаторов видимости***.
- Реализует ограничение доступа к атрибутам и операциям со стороны других объектов.


Телевизор
+ Язык экранного меню
— Частота каналов
+ Порядок и именование каналов
+ ...
— Самодиагностика()
+ Включить()
+ Выключить()
+ Поиск каналов()
— Декодирование сигнала()
+ Переключение каналов()
+ ...()

```
class A
{
 public:
   int a, b; //данные открытого интерфейса
   int ReturnSomething(); //метод открытого интерфейса
 private:
   int Aa, Ab; //скрытые данные
   void Do_Something(); //скрытый метод
};
```

Класс A инкапсулирует свойства Aa, Ab и метод Do_Something(), представляя внешний интерфейс ReturnSomething, a, b.

Видимость в языке UML обозначается с помощью квантора видимости *(visibility)*

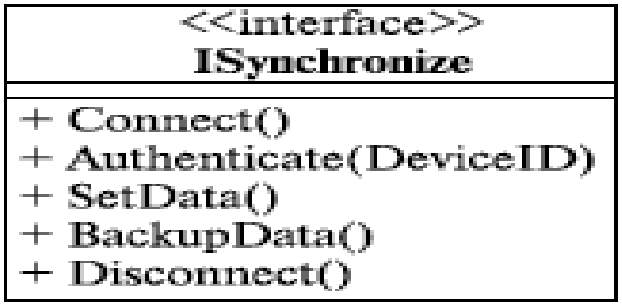
- Открытый (***public***). *Атрибут виден всем остальным классам. Любой класс, связанный с данным в рамках диаграммы или пакета, может просмотреть или изменить значение атрибута. Обозначается символом* ***«+»*** *перед именем атрибута.*
- Защищенный (***protected***). *Любой потомок данного класса может пользоваться его защищенными свойствами. Обозначается* ***«#»***
- Закрытый (**private**). Атрибут с этой областью видимости недоступен или не виден для всех классов без исключения. Обозначается **«-»**
- Пакетный (**package**). Атрибут является открытым, но только в пределах своего пакета. обозначается **«~».**

Интерфейс –

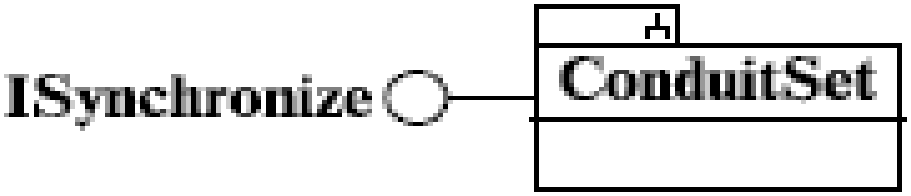
стандартный способ доступа к объекту,

отражает внешние проявления объекта

Интерфейс всегда реализуется некоторым классом



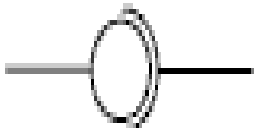
1 способ



2 способ

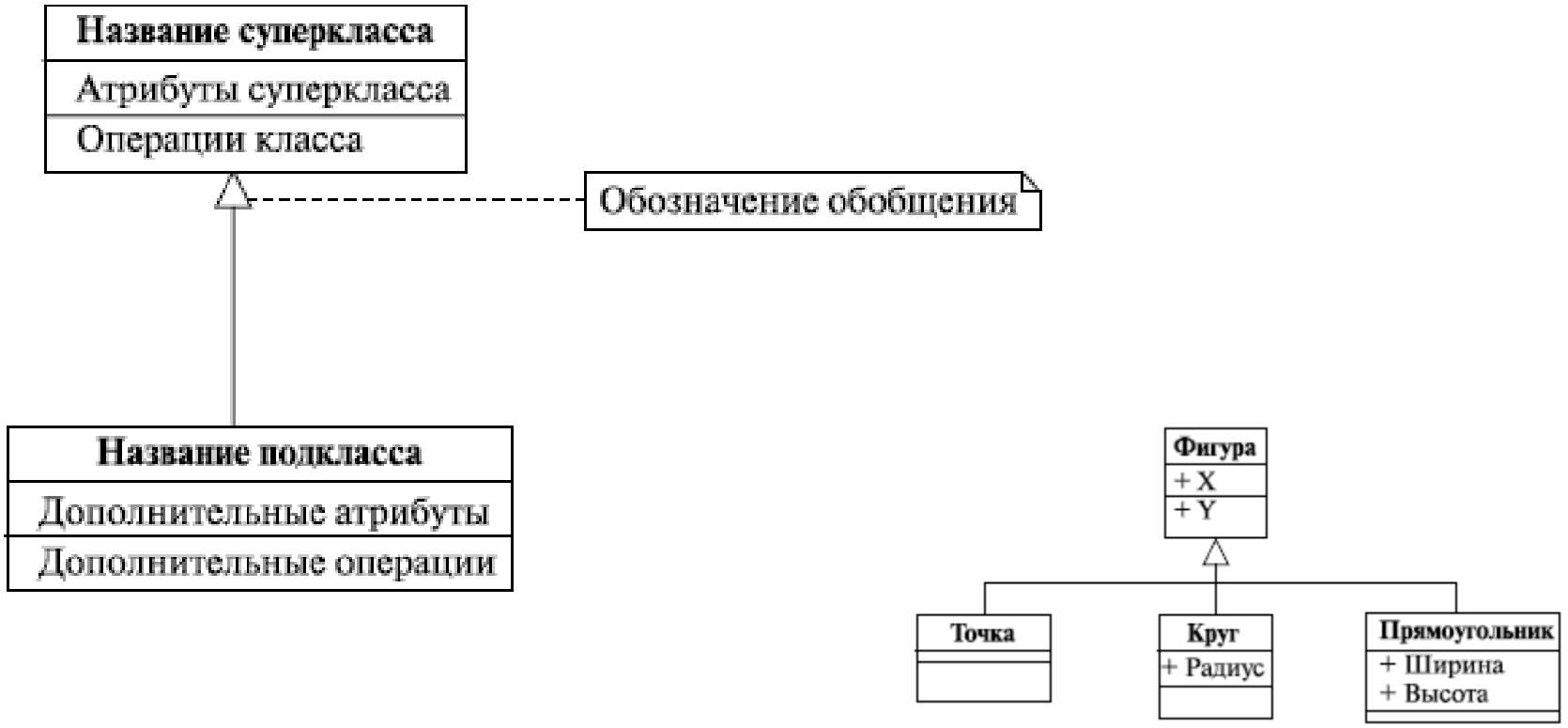


3 способ



Обобщение (генерализация, наследование) - это *отношение* между более общей сущностью, называемой ***суперклассом***, и ее конкретным воплощением, называемым ***подклассом***.

При этом все атрибуты и *операции* суперкласса независимо от *модификаторов видимости* входят в состав подкласса.



Полиморфизм – способность объекта использовать методы производного класса, которых не существовало на момент создания базового.

Возможность объектов с одинаковой спецификацией иметь разную реализацию
Полиморфизм является основой для реализации *механизма интерфейсов*

Связи

Связь представляет собой семантическую взаимосвязь между классами.

Существуют четыре типа связей, которые могут быть установлены между классами:

ассоциации, зависимости, агрегации и обобщения.

Ассоциации

Ассоциация (association) – это семантическая связь между классами.



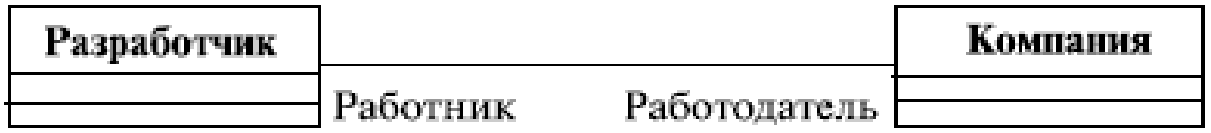
Связи

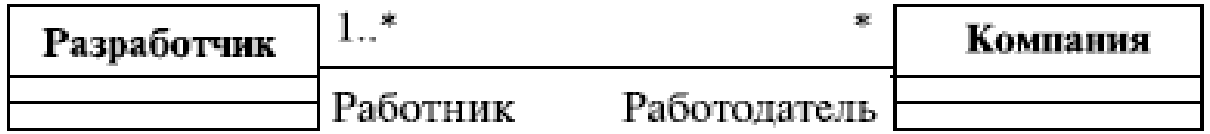
Ассоциация может иметь имя, показывающее природу отношений между объектами, при этом в имени может указываться **направление чтения** связи при помощи треугольного маркера. Однонаправленная *ассоциация* может изображаться стрелкой.





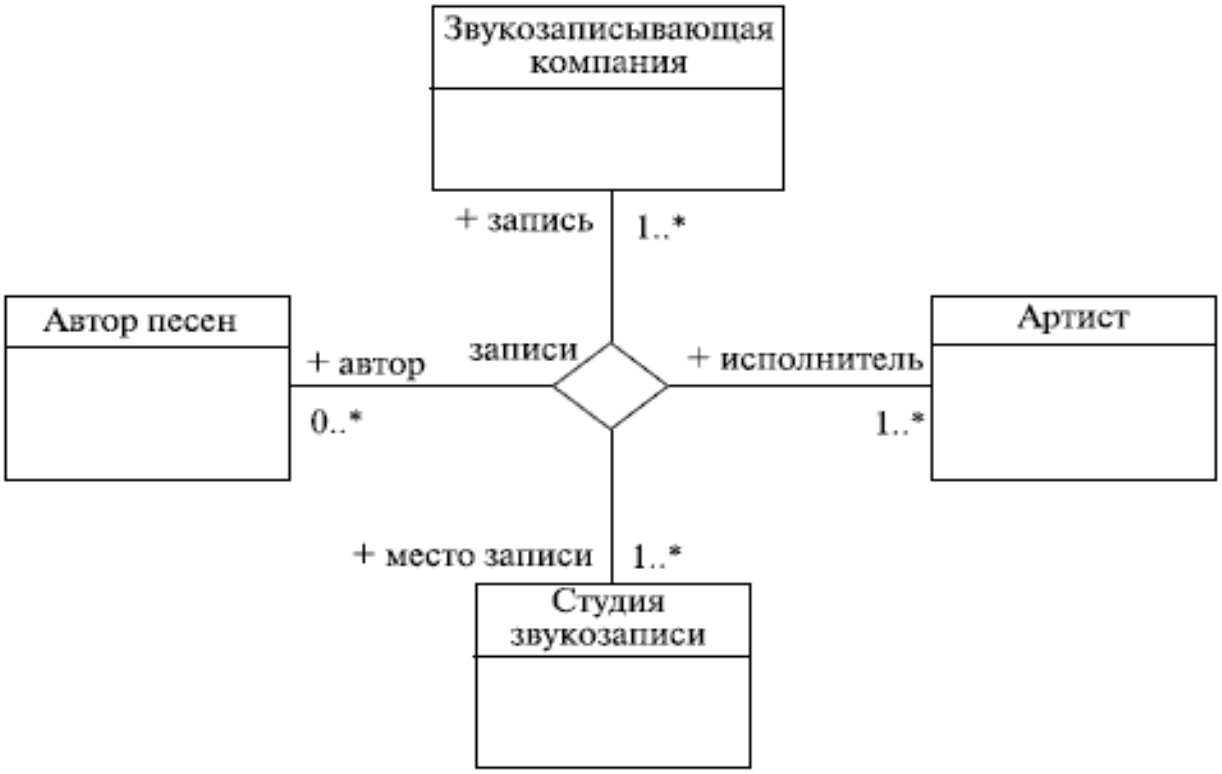
На диаграмме можно указать ***роли***, которые каждый *класс* играет в данном отношении, и ***кратность***, то есть количество объектов, связанных отношением





Связи

Ассоциация может объединять три и более класса. В этом случае она называется **n-арной** и изображается ромбом на пересечении линий



Связи

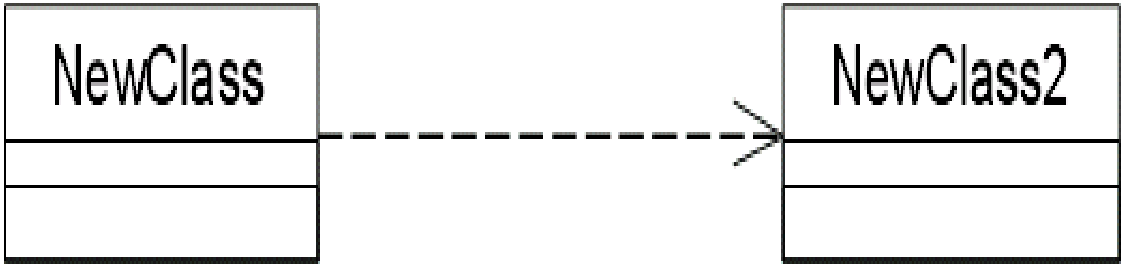
В отношении между двумя классами сама *ассоциация* тоже может иметь свойства и, следовательно, тоже может быть представлена в виде класса.



Связи

Зависимости

Связи зависимости (dependency) также отражают связь между классами, но они всегда однонаправлены и показывают, что один класс зависит от определений, сделанных в другом.



Агрегации

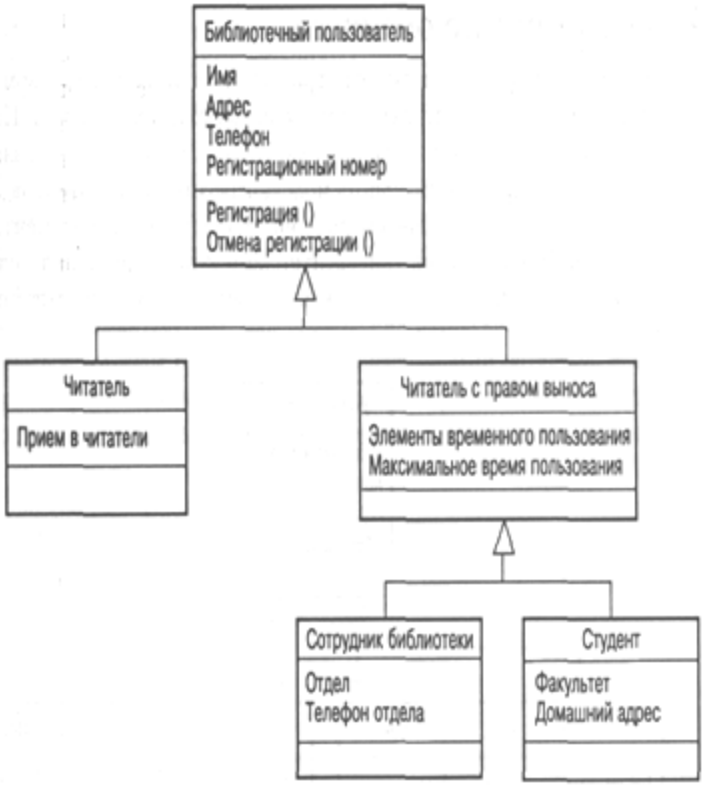
Агрегации (aggregations) представляют собой более тесную форму ассоциации. Агрегация – это связь между целым и его частью.





Обобщения (Наследование)

Обобщение (наследование) - это отношение типа общее-частное между элементами модели



Множественность

Множественность (multiplicity) показывает, сколько экземпляров одного класса взаимодействуют с помощью этой связи с одним экземпляром другого класса в данный момент времени.

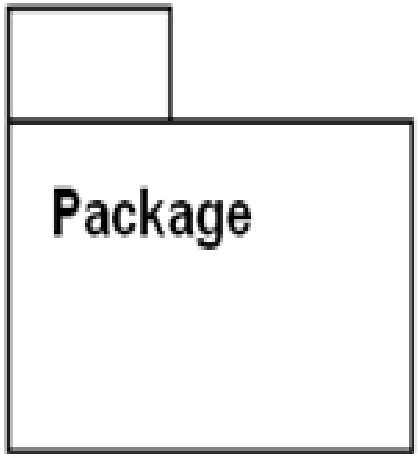
Множественность	Значение
0..*	Ноль или больше
1..*	Один или больше
0..1	Ноль или один
1..1 (сокращенная запись: 1)	Ровно один

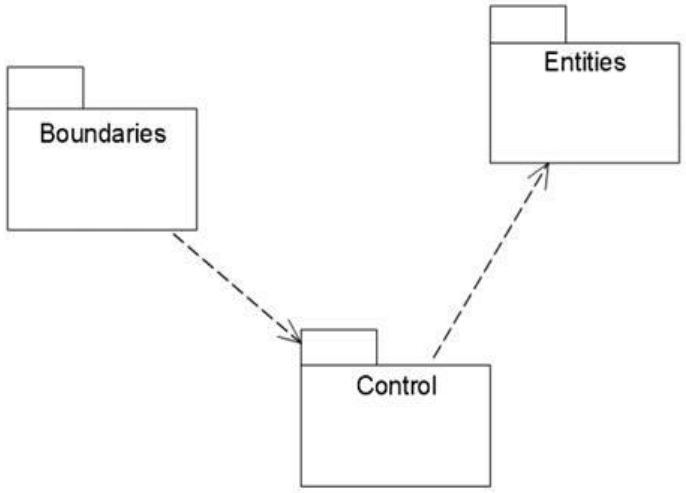
Пакет.

Механизм пакетов

В контексте диаграмм классов, пакет - это вместилище для некоторого набора классов и других пакетов.

Пакет является самостоятельным пространством имен.





Пример диаграммы пакетов

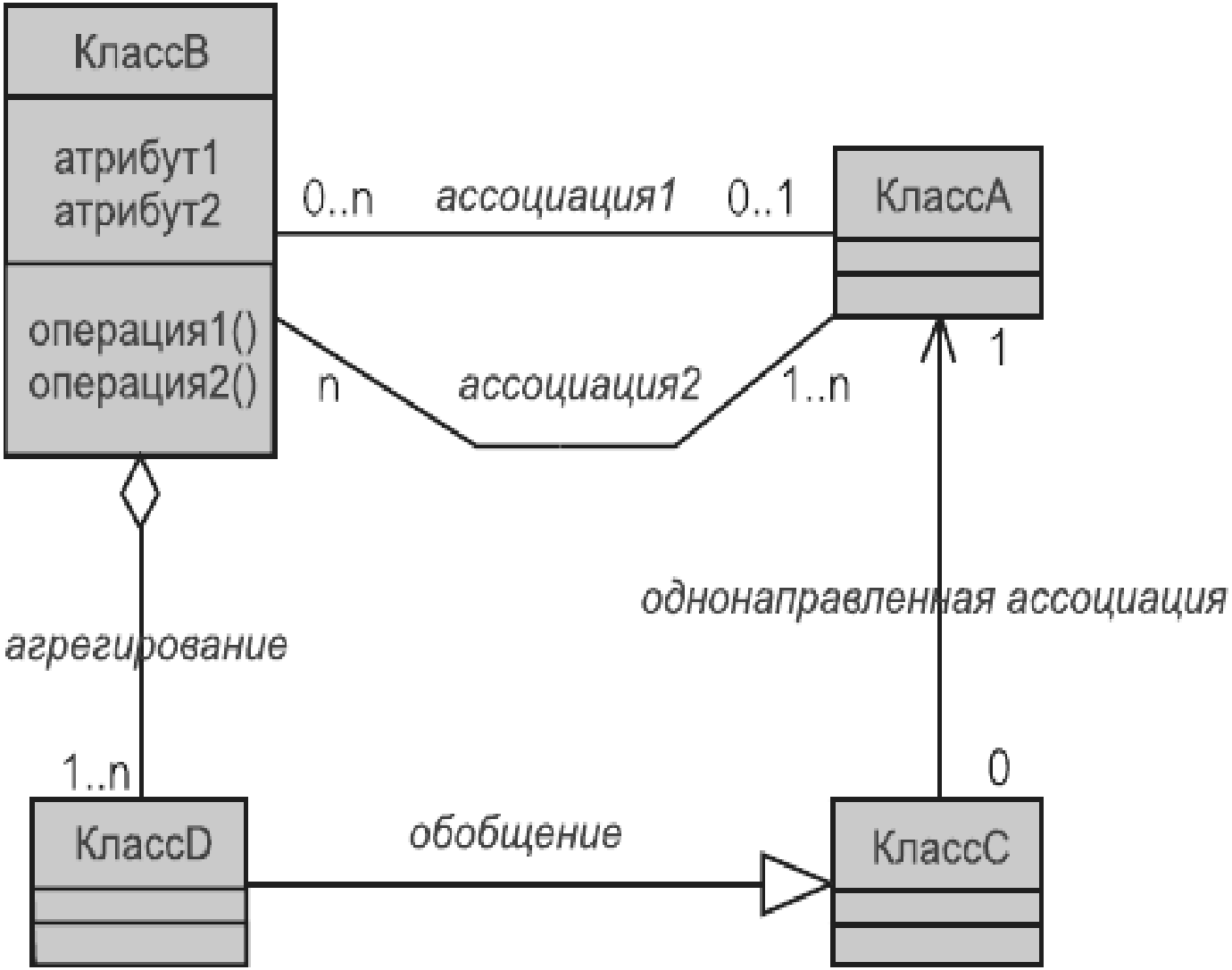
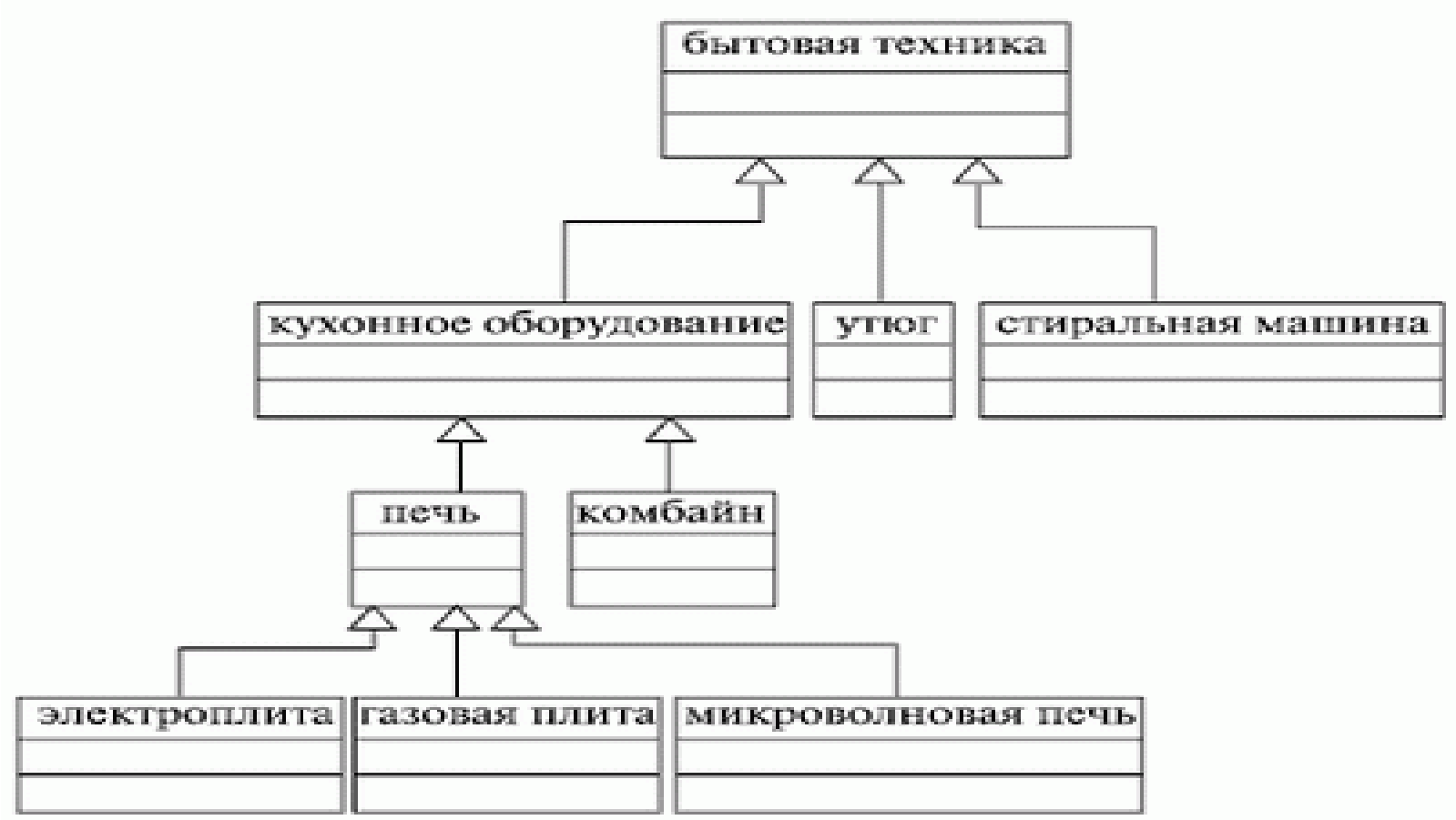
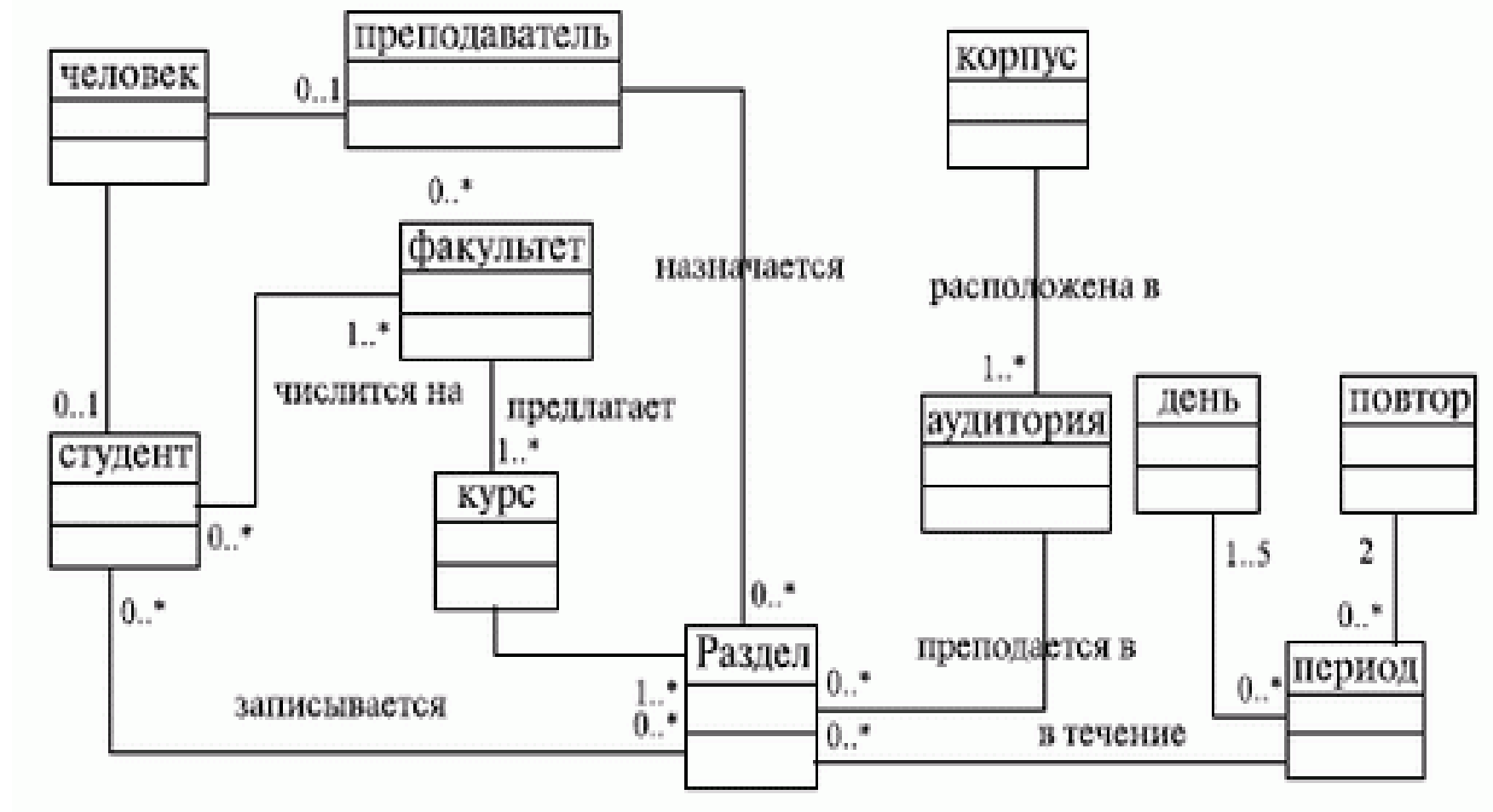


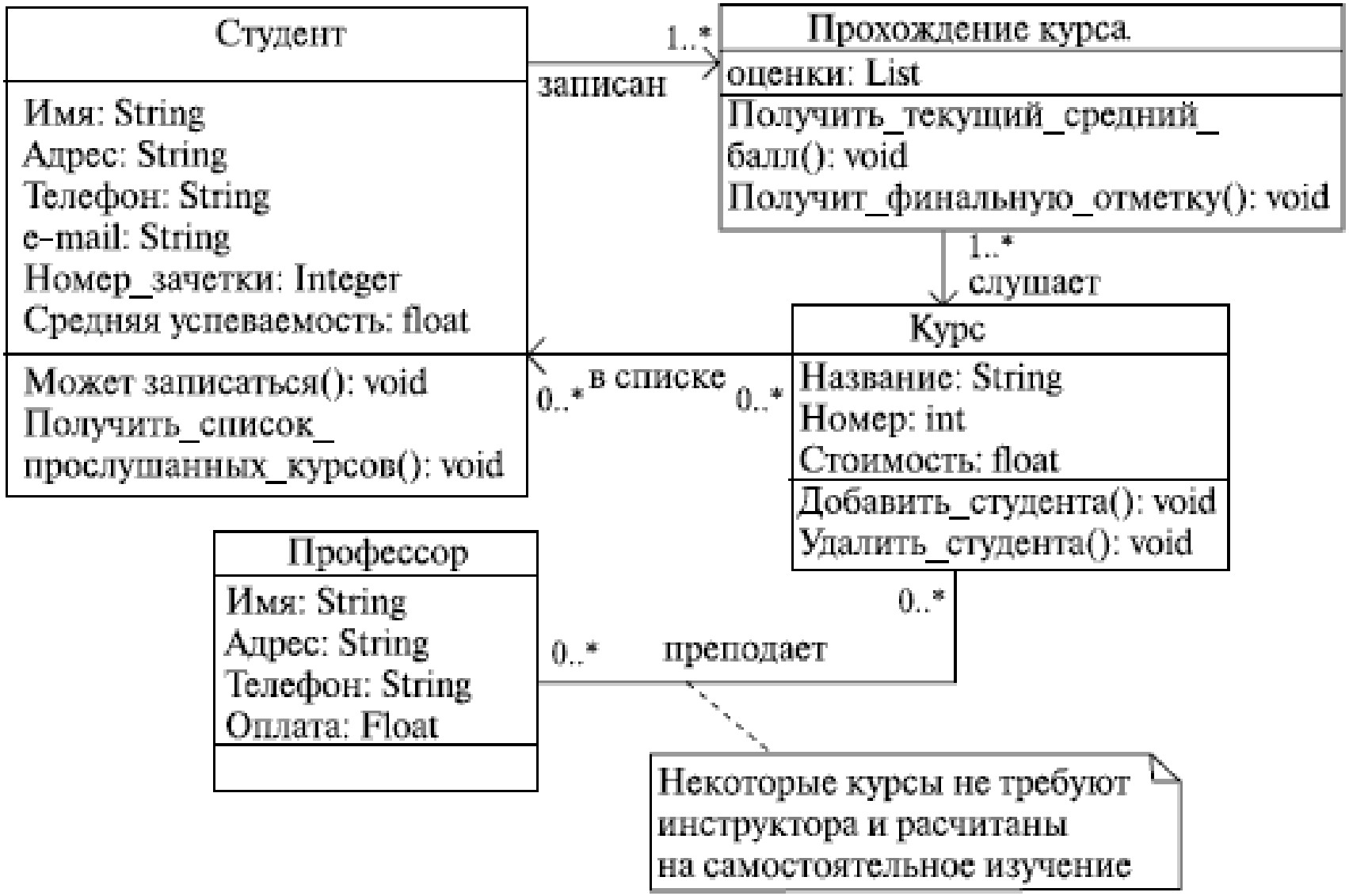
Рис. 2.6. Элементы моделирования диаграммы классов

Пример иллюстрирует с помощью операции наследования или генерализации «генеалогическое древо» бытовой техники



Пример описывает предметную область задачи об автоматизации работы ОУ





На этой диаграмме кроме **кратности на концах связей** обозначены **свойства** (и их типы) и **операции**

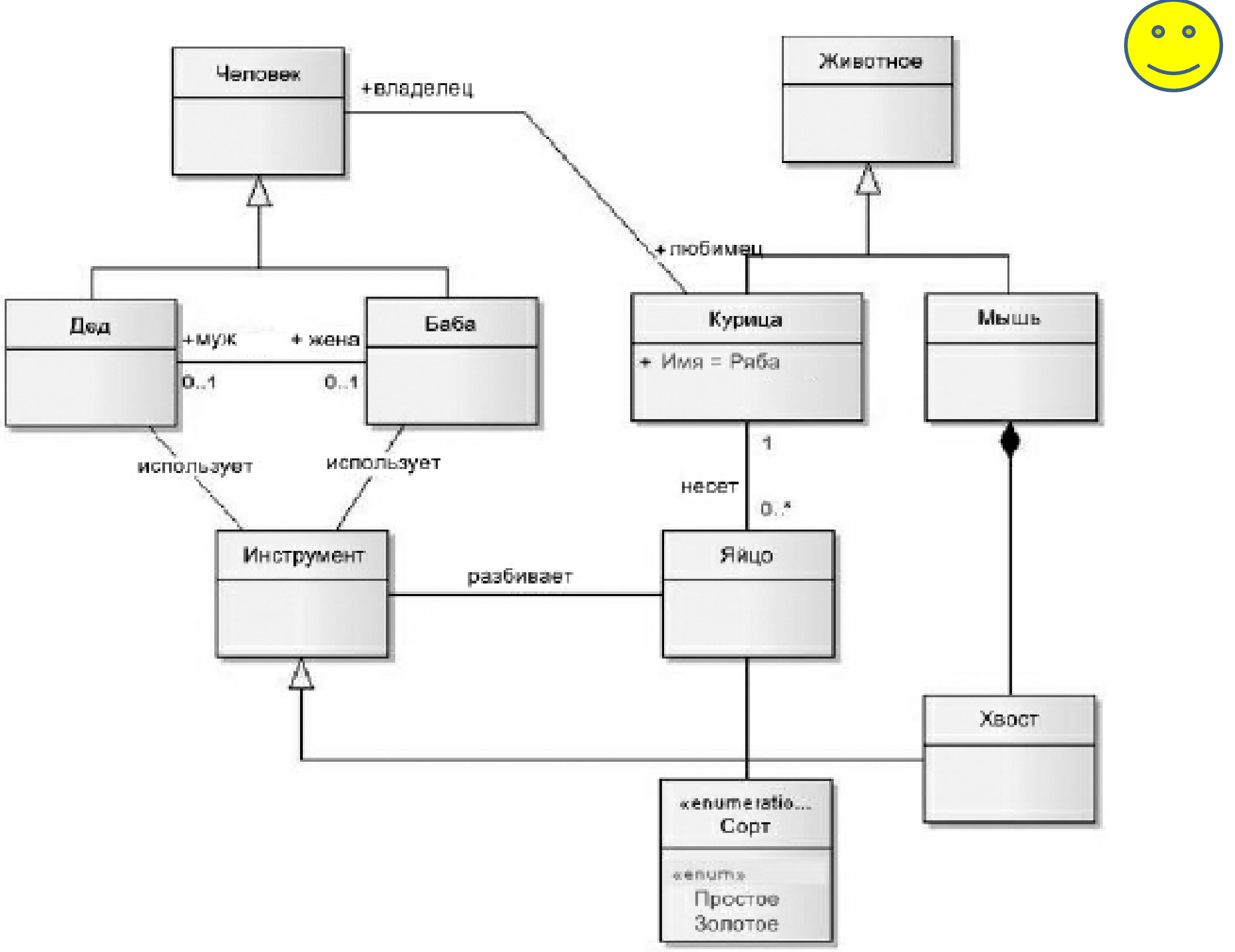

Человек
+владелец
Животное
+любимец
Дед
+муж
0..1
+ жена
0..1
Баба
Курица
+ Имя = Ряба
Мышь
1
несет
0..*
используует
используует
Инструмент
разбивает
Яйцо
Хвост
«enumeratio...
Сорт
«enum»
Простое
Золотое

Диаграмма объектов (object diagram)

Объект (object) - экземпляр класса

Объект (object) -

- конкретная материализация абстракции;
- сущность с хорошо определенными границами, в которой инкапсулированы состояние и поведение;
- экземпляр класса. Объект уникально идентифицируется значениями атрибутов, определяющими его состояние в данный момент времени.

Объект, *как и класс, обозначается прямоугольником, но его **имя** подчеркивается*.

Под словом **имя** здесь мы понимаем **название объекта** и наименование его класса, разделенные двоеточием.





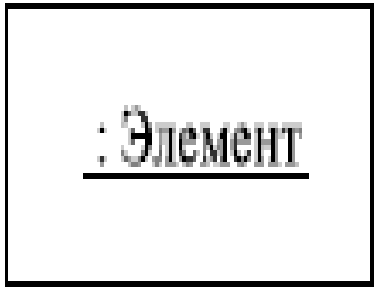


Диаграмма объектов *- это своего рода снимок состояния системы в определенный момент времени*, показывающий множество объектов, их состояния и отношения между ними в данный момент.

Диаграмма объектов используется для пояснения и детализации диаграмм взаимодействия, например, диаграмм последовательностей



Некоторая фирма "раскручивает" новый товар или услугу.
В этом процессе участвуют вице-президент по маркетингу, вице-президент по продажам, менеджер по продажам, торговый агент, специалист по рекламе, некое печатное издание и покупатель.
Показано кто с кем взаимодействует.
все объекты анонимные!

Диаграмма объектов учебной среды "Робот" для Turbo Pascal

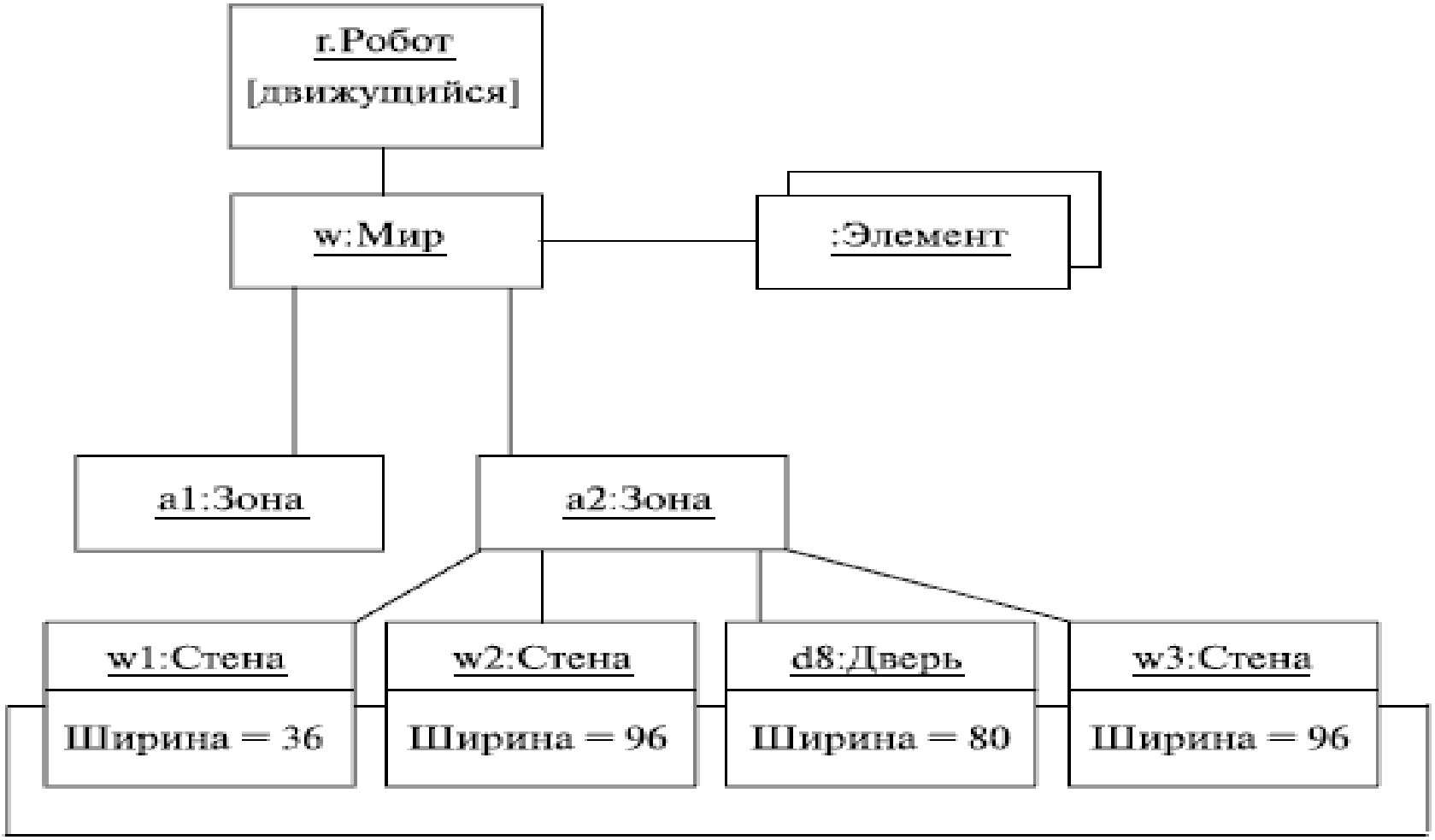
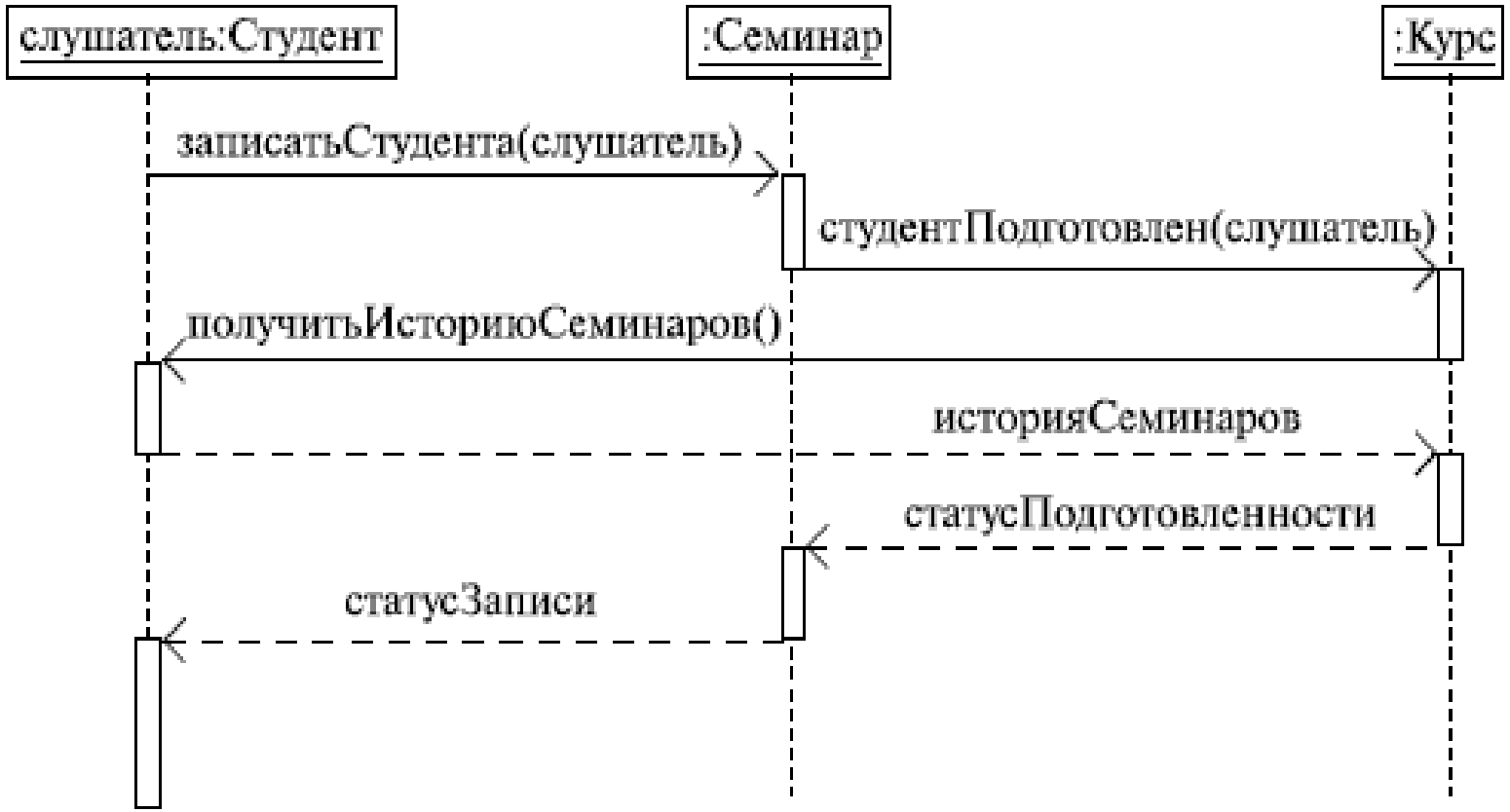


Диаграмма последовательностей (sequence diagram)

Диаграмма последовательностей *отображает взаимодействие объектов* ***в динамике***.

т. е. временные особенности передачи и приема сообщений объектами.



Студент хочет записаться на некий семинар, предлагаемый в рамках некоторого учебного курса.

С этой целью проводится проверка подготовленности студента, для чего запрашивается список (история) семинаров курса, уже пройденных студентом (перейти к следующему семинару можно, лишь проработав материал предыдущих занятий)

После получения истории семинаров объект класса "Слушатель" получает статус подготовленности, на основе которой студенту сообщается результат (статус) его попытки записи на семинар.

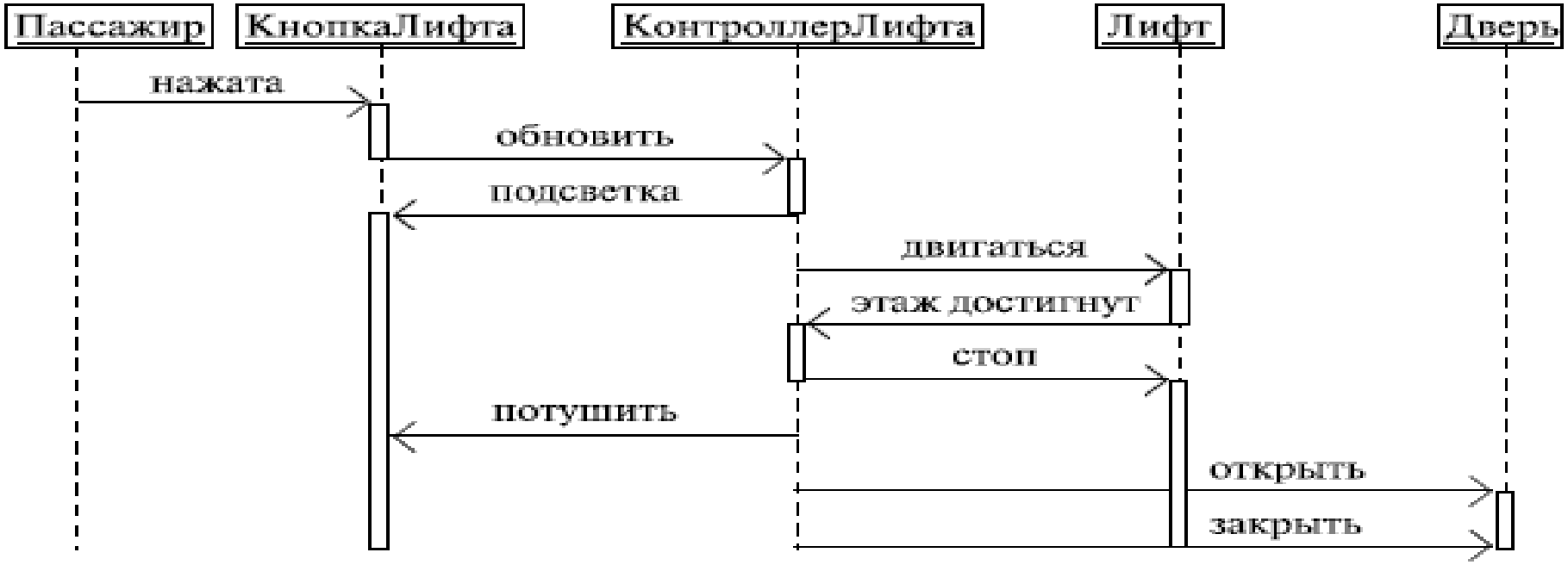

Пассажир
КнопкаЛифта
КонтроллерЛифта
Лифт
Дверь
нажата
обновить
подсветка
двигаться
этаж достигнут
стоп
потушить
открыть
закрыть

Диаграмма взаимодействия (кооперации, collaboration diagram)

Диаграмма взаимодействия *показывает* ***поток сообщений между объектами системы и основные ассоциации между ними*** и по сути, является альтернативой диаграммы последовательностей.



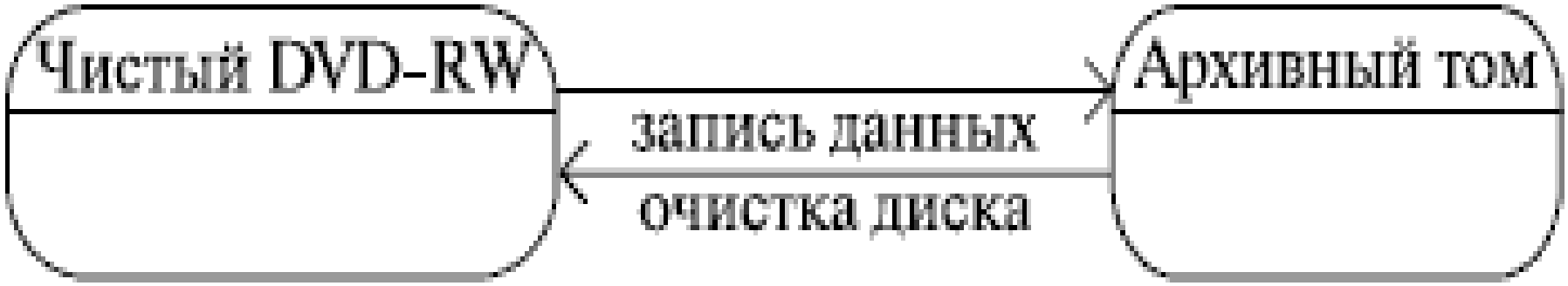
Диаграмма описывает работу персонала библиотеки по обслуживанию клиентов: библиотекарь получает заказ от клиента, поручает сотруднику найти информацию по нужной клиенту книге, а после получения данных поручает еще одному сотруднику выдать книгу клиенту.


:Пользователь
курс: Курс
1..*
1: УправлениеКурсом
2: СоздатьКурс
0..*
Администратор:
АдминистраторКурса
3: СоздатьРаздел
раздел: Раздел
4: НазначитьНаКурс
преподаватель:
Преподаватель

Диаграмма состояний (statechart diagram)

Состояние (state) - ситуация в жизненном цикле объекта, во время которой он удовлетворяет некоторому условию, выполняет определенную деятельность или ожидает какого-то события. Состояние объекта определяется значениями некоторых его атрибутов и присутствием или отсутствием связей с другими объектами.

Диаграмма состояний показывает, как объект переходит из одного состояния в другое.


Чистый DVD-RW
Архивный том
запись данных
очистка диска

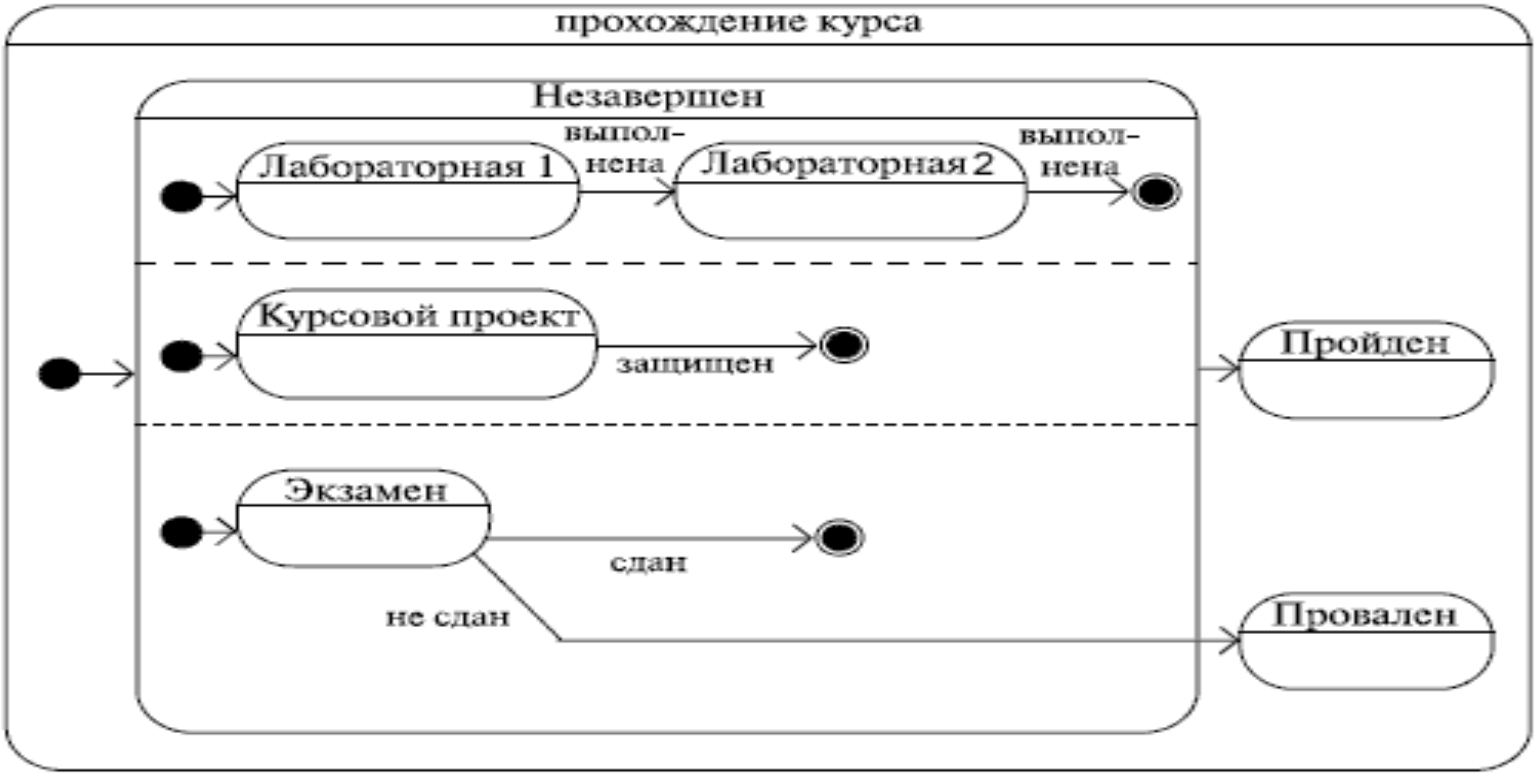
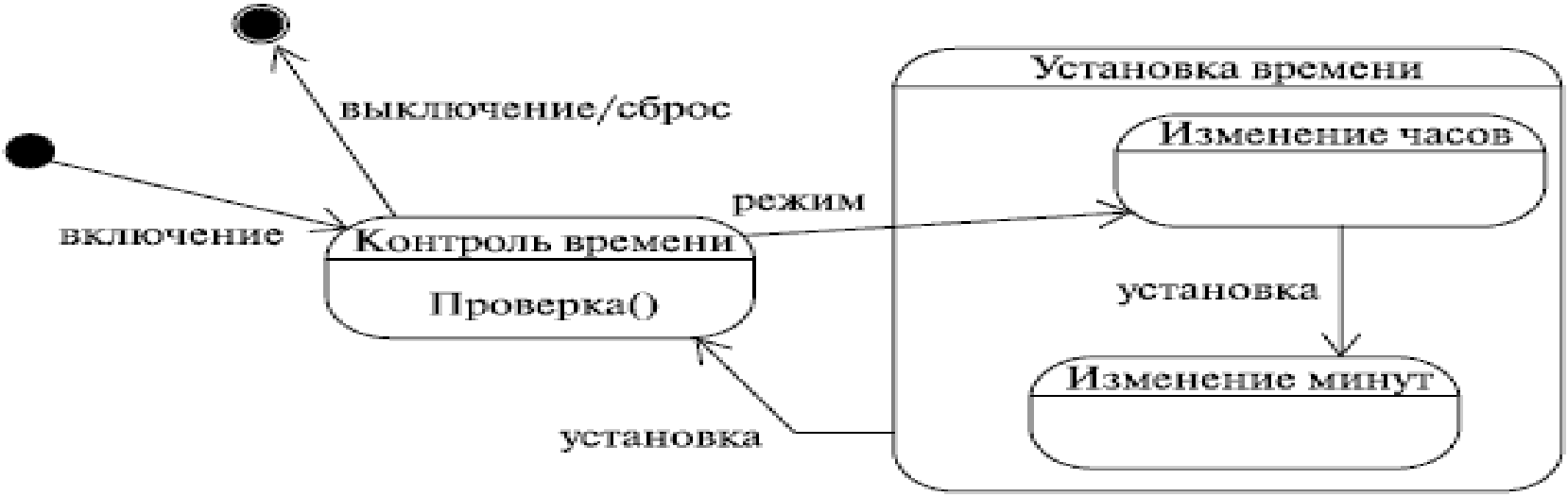


Диаграмма прохождения академического курса студентом
Здесь присутствует ***составное состояние***, включающее другие состояния, одно из которых содержит также параллельные *подсостояния*.
Для того чтобы пройти курс, студент должен выполнить лабораторные работы, защитить курсовой проект и сдать экзамен.



это устройство - таймер!

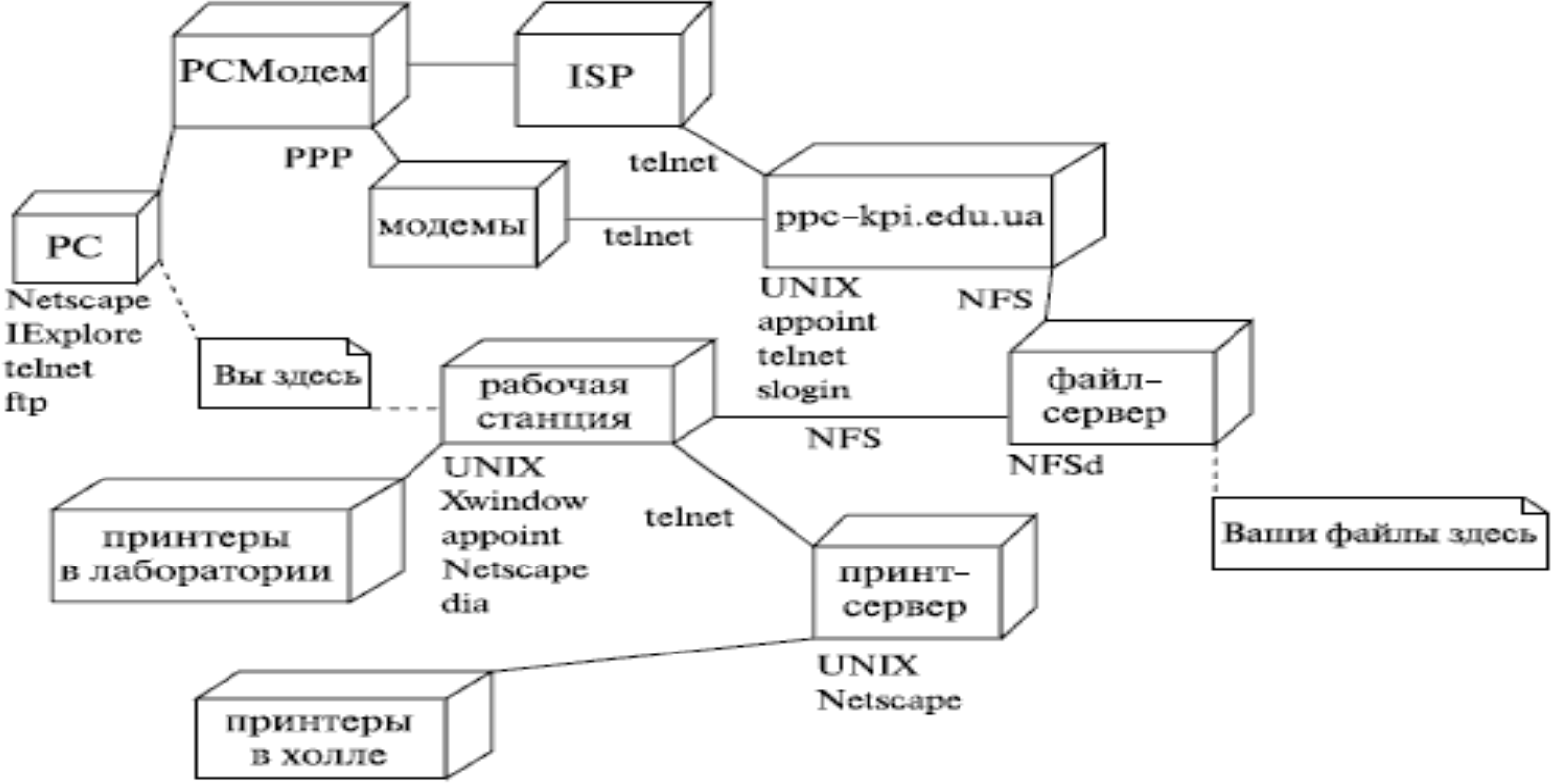
Такой прибор может применяться в составе различных реле, например, для отключения телевизора по истечении указанного промежутка времени. Основное его назначение - контроль времени (проверка, не истек ли указанный промежуток), но у него есть еще один режим работы - установка. По истечении указанного промежутка времени или при "сбросе" устройство отключается.

Диаграмма развертывания (deployment diagram)

графическое представление ИТ-инфраструктуры

Диаграмма развертывания показывает топологию системы и распределение компонентов системы по ее узлам, а также соединения - маршруты передачи информации между аппаратными узлами.


<<База данных>>
БД Заказчики
<<База данных>>
БД Продажи
<<Сервер приложений>>
Продажи и обслуживание
<<Клиент>>
Экран обслуживания клиента



Это инфраструктура некоего учебного заведения, включающая шлюз, файл-сервер, принт-сервер, принтеры в лабораториях и холле и т. д.
Пользователь может получить доступ к этим ресурсам либо со своей домашней машины, либо с рабочих станций, находящихся в лабораториях вуза.
подписи под линиями, показывающими линии передачи информации (например, видно, что рабочая станция получает доступ к файлам, хранящимся на файл-сервере, посредством NFS)

-  Диаграмма классов (Class diagram)
  Диаграмма классов - визуальное отображение различных статических отношений между класс-подобными элементами. Диаграмма классов может содержать не только классы, но также и интерфейсы, перечислимые типы, пакеты, различные отношения, инстанции и их связи.
-  Диаграмма прецедентов (Use case diagram)
  Диаграмма прецедентов - отображение отношений между вариантами использования (прецедентами) определенной системы или объекта и внешними акторами. Вариант использования отображает функции системы и то, как эти функции взаимодействуют с внешними акторами.
-  Диаграмма сообщений (Sequence Diagram)
  Диаграмма сообщений отображает взаимодействие инстанций. Она является прямым отображением множества взаимных воздействий (InteractionInstanceSet) между элементами множества инстанций (CollaborationInstanceSet). В то время как Диаграмма сообщений роли ориентирована на классификаторы-роли, обычная Диаграмма сообщений - на инстанции.
-  Диаграмма сообщений роли (Sequence Role Diagram)
  Диаграмма сообщений роли отображает взаимодействия в концепции ролей. Она является прямым отображением Interaction (множества взаимных сообщений между классификаторами-ролями) в пределах Collaboration. В то время как Диаграмма сообщений - отображение инстанций, Диаграмма сообщений роли - отображение классификаторов-ролей.
-  Диаграмма коллаборации (Collaboration Diagram)
  Диаграмма коллаборации отображает взаимодействие между инстанциями. Она является прямым отображением модели взаимодействия инстанций, входящих в

CollaborationInstanceSet. В то время как диаграмма коллаборации ролей - отображение классификаторов-ролей, обычная диаграмма коллаборации - отображение инстанций.

-  Диаграмма коллаборации ролей
  Диаграмма коллаборации ролей отображает взаимодействия между ролями. Она является прямым отображением модели взаимодействия классификаторов-ролей внутри коллаборации. В то время как обычная диаграмма коллаборации ориентирована на отображение инстанций, диаграмма коллаборации ролей - отображение классификаторов-ролей.
-  Диаграмма состояний (Statechart Diagram)
  Диаграмма состояний выражает статическое поведение определенного объекта через состояния и переходы состояний. Хотя диаграмма состояний обычно используется, чтобы выразить поведение инстанций классов, она может также использоваться, чтобы выражать поведение и других элементов.
-  Диаграмма действий (Activity Diagram)
  Диаграмма действий - специальная форма диаграммы состояний, которая является подходящей для того, чтобы отображать поток выполнения действий. Диаграмма действий в общем случае используется для отображения любых потоков обработки, но чаще всего применительно к объектам подобным классам, пакетам и операциям.
-  Диаграмма компонентов (Component Diagram)
  Диаграмма компонентов отображает зависимость между программными компонентами. Элементы, которые составляют программные компоненты и элементы, которые реализуют эти компоненты, могут быть отображены на диаграмме компонентов.
-  Диаграмма развертывания (Deployment Diagram)
  Диаграмма развертывания отображает аппаратные элементы компьютера, другие устройства и программные компоненты, а также процессы и объекты, которые им назначены.
-  Композиционная структурная диаграмма (Composite Structure Diagram)
  Композиционная структурная диаграмма - диаграмма, выражающая внутреннюю структуру классификатора. Она показывает его точки зрения взаимодействия с другими частями системы.